Guía de actividades

A continuación, se presentan las orientaciones para las actividades de aprendizaje de la unidad de **Procesamiento distribuido.**

Generalidades

Tiempo total previsto para el desarrollo de las actividades: 16.5 horas.

Resultados de aprendizaje:

- Utilizar eficazmente herramientas y software especializados en analítica de datos para procesar, analizar y visualizar grandes volúmenes de datos en diferentes contextos profesionales."

Indicadores de cumplimiento

- Implementar pipelines de procesamiento de datos utilizando eficazmente PySpark en Google Colab y Databricks Community Edition, aplicando técnicas de optimización (particionamiento, formatos eficientes, broadcast joins) y creando visualizaciones apropiadas para grandes volúmenes de datos."

Actividades

Actividad 1: Análisis comparativo de herramientas. Selección tecnológica justificada para el proyecto. (formativo)
Actividad 2: Investigación y diseño de arquitectura con *PySpark (evaluativa)*

Actividad 1: Análisis comparativo de herramientas

Los estudiantes realizarán un análisis **teórico y práctico** de herramientas de procesamiento de datos, combinando investigación, experimentación y validación técnica. El objetivo es fundamentar decisiones tecnológicas para su proyecto y demostrar la viabilidad de una arquitectura escalable mediante un ***Proof of Concept (PoC)*** *en PySpark.*

Fase 1. Investigación

1. Benchmark comparativo de herramientas

Comparación empírica de rendimiento entre Pandas, *Polars y PySpark* usando los mismos 3 archivos mensuales de la Hoja de ruta. El *benchmark* incluirá medición automatizada de tiempo de ejecución y uso de memoria mediante la función *measure_performance*(), seguido de una matriz de decisión tecnológica ponderada con criterios específicos.

Metodología:

1. Configurar entorno de prueba en Google Colab con medición de tiempo: import time, import psutil

2. Cargar 3 archivos CSV de emergencias (de preferencia, los mismos seleccionados en la Hoja de ruta) con cada herramienta

3. Ejecutar operaciones específicas en cada herramienta:
- Carga: leer 3 archivos CSV y combinar en único DataFrame
- Filtrado: filtrar solo emergencias de "USAQUEN" y "CHAPINERO"
- Agregación: contar emergencias por tipo para datos filtrados
- Join temporal: unir con tabla de códigos de emergencia
- Exportación: guardar resultado filtrado en formato parquet
- Medir tiempo total y tiempo por operación
- Medir tiempo de ejecución para cada operación usando time.time()

4. Registrar uso de memoria con psutil.virtual_memory()

5. Documentar complejidad de código (líneas necesarias por operación)

Herramientas:

```
import time

import psutil

start_time = time.time()

# [operación a medir]

execution_time = time.time() - start_time

memory_usage = psutil.virtual_memory().percent
```

2. Matriz de decisión tecnológica

Desarrollo de matriz de evaluación multicriterio para seleccionar la herramienta idónea según el caso de uso específico del equipo. Evaluación de factores técnicos, recursos disponibles y requerimientos del proyecto.

Metodología:

- Definir criterios y pesos: rendimiento (30%), facilidad de uso (25%), escalabilidad (20%), recursos (15%), curva de aprendizaje (10%).
- Puntuar de 1 a 5 cada herramienta según los resultados del benchmark.
- Calcular puntaje ponderado: Suma(criterio_i × peso_j) para cada herramienta
- Considerar las restricciones de Colab (RAM 12 GB, tiempo 12 h).
- Validar la elección con el caso de uso del equipo.

3. Selección tecnológica justificada

Decisión final fundamentada sobre la herramienta principal y complementarias a utilizar en el proyecto, con plan de implementación específico y consideración de limitaciones técnicas.

Metodología:

- Revisar benchmark y matriz de decisión.
- Confirmar viabilidad técnica.
- Definir herramienta principal (obligatoria: **PySpark**) y complementarias.
- Establecer criterios de uso según el volumen de datos.
- Diseñar un plan de implementación por fases.

Actividad 2. Diseño de arquitectura y PoC en PySpark (evaluativa)

Esta evaluación permitirá a los estudiantes demostrar su capacidad para diseñar arquitecturas escalables y desarrollar prototipos funcionales utilizando tecnologías de procesamiento distribuido. Los equipos deben traducir el análisis comparativo realizado durante la investigación en decisiones arquitecturales concretas y validar su viabilidad técnica mediante un proof of concept implementado.

Nota: Recordar que los desarrollos son progresivos, por lo que los datos requeridos en este entregable corresponden a los definidos en la sección “b. Investigación”.

1. Documento de arquitectura técnica

Elaboración de documento técnico estructurado describiendo la arquitectura completa del sistema de análisis de emergencias 123, incluyendo flujos de datos, componentes y decisiones de diseño justificadas.

Metodología:

- Crear un diagrama (*draw.io o Lucidchart*) con ingesta, procesamiento, almacenamiento y salida.
- Documentar decisiones técnicas y configuraciones de *PySpark.*
- Incluir estrategia de particionamiento (mes/localidad).
- Definir métricas de monitoreo y KPIs.

Herramientas:

1. Template de documentación: Estructura predefinida con secciones: introducción, objetivos, arquitectura propuesta, componentes técnicos, flujos de datos, decisiones de diseño, riesgos y mitigaciones

2. Herramientas de diagramación: Draw.io o Lucidchart

Entregables:

1. Documento técnico (.docx) de 5 a 8 páginas con: diagrama de arquitectura, justificación de decisiones, configuraciones específicas, estimación de recursos
2. Diagrama de arquitectura (.png) de alta resolución exportado independientemente

2. Proof of Concept (PoC) funcional en PySpark

Implementación de pipeline básico en PySpark que procese los 3 meses de datos de emergencias 123 seleccionados, demostrando ingesta, transformación básica, validación de calidad y exportación de resultados.

Metodología:

- Configurar PySpark en Google Colab
- Implementar ingesta de 3 archivos CSV desde Google Drive
- Aplicar transformaciones básicas que sean requeridas: limpieza de nulos, conversión de tipos, estandarización de códigos, etc.
- Implementar validación de calidad: conteo de registros, verificación de rangos de fechas, consistencia de códigos
- Exportar resultado a formato Parquet para demostrar optimización

Herramientas:

```
# Configuración PySpark básica para Colab
!pip install pyspark
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("PoC_Emergencias123_Bogota") \
    .config("spark.driver.memory", "4g") \
    .config("spark.driver.maxResultSize", "2g") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.adaptive.coalescePartitions.enabled", "true") \
    .config("spark.sql.adaptive.skewJoin.enabled", "true") \
    .config("spark.sql.execution.arrow.pyspark.enabled", "true") \
    .config("spark.sql.adaptive.advisoryPartitionSizeInBytes", "64MB") \
    .getOrCreate()
```

Entregables:

- Notebook de Google Colab (.ipynb) funcional con código documentado y outputs visibles
- Archivo Parquet resultante (.parquet) subido a Google Drive como evidencia
- Log de validación (.txt) con métricas de calidad: registros procesados, errores encontrados, tiempo de ejecución

3. Mini-pitch técnico (5 minutos)

Presentación técnica estructurada donde cada equipo defiende sus decisiones arquitecturales y demuestra el PoC funcionando.

Metodología:

- Preparar presentación de máximo 5 minutos con: 1 minuto para la introducción, 2 minutos para la demostración de arquitectura, 1,5 minutos para el demo en vivo del PoC, 0,5 minutos para conclusiones (estos valores son de referencia y aproximados).
- Grabar video en el cual se presenten las secciones relacionadas en el ítem 1

Herramientas especificas:

- Plantilla de presentación: PowerPoint o herramienta de preferencia con máximo 6 slides: título, arquitectura, decisiones técnicas, demo en vivo, métricas de rendimiento, conclusiones
- Demo en vivo: Notebook con celdas pre-ejecutadas y tiempo de ejecución

Entregable: Video de demostración (.mp4) de máximo 5 minutos

4. Métricas de rendimiento

Documentación cuantitativa del desempeño del PoC con mediciones específicas de tiempo, memoria, throughput y comparación con benchmarks establecidos en la fase de investigación**.**

Metodología:

1. Ejecutar PoC 3 veces para obtener métricas promedio. Métricas obligatorias: Tiempo total (segundos), memoria pico (%), throughput (registros/minuto), tasa de éxito de validaciones (%)
2. Medir tiempo end-to-end: desde carga hasta exportación de resultados
3. Registrar uso de memoria pico durante procesamiento
4. Calcular throughput: registros procesados por minuto
5. Comparar con métricas de Pandas del benchmark inicial

Entregables:

- Reporte de métricas (.xlsx) con mediciones de 3 ejecuciones y promedios
- Gráfico comparativo (.png) que evidencia mejora en relación con Pandas en tiempo y memoria
- Análisis de escalabilidad (.docx) de 1 página con proyección teórica para volúmenes de 1 a 2 años de datos. Basándose en las métricas obtenidas con 3 meses, estimar recursos necesarios y puntos de quiebre para escalabilidad.

Recomendaciones para la realización

Fechas: cumplir con las fechas establecidas para la entrega de la actividad para asegurar una

experiencia de aprendizaje óptima.

Importancia de la actividad: esta actividad es evaluativa, por lo que es crucial realizar y entregar el trabajo.

Revisión de la rúbrica: consultar la rúbrica al final del documento para verificar los criterios solicitados por el profesor.

Rúbrica de evaluación

Criterios de evaluación	Indicadores de cumplimiento									
	Excelente		Bueno		Necesita mejorar		Deficiente		No cumple con el criterio o no entrega	
	pts.	Ítem	pts.	Ítem	pts.	Ítem	pts.	Ítem	pts.	Ítem
Implementación correcta del benchmark empírico	1.5	Benchmark ejecuta sin errores con medición automatizada de tiempo y memoria usando measure_performance(). Incluye 3 herramientas (Pandas, Polars, PySpark) con mismo dataset de 3 meses. Operaciones comparadas: carga, filtrado, agregación, joins. Mediciones consistentes y repetibles (múltiples iteraciones).	1.2	Benchmark funcional con medición básica de rendimiento. Incluye las 3 herramientas principales con operaciones comparativas estándar. Mediciones correctas, pero con menor profundidad metodológica.	0.8	Benchmark básico que ejecuta, pero con metodología de medición inconsistente. Falta alguna herramienta u operación. Mediciones parcialmente confiables.	0.4	Benchmark incompleto con errores en la medición o falta de herramientas importantes. Metodología deficiente.	0	No presenta benchmark funcional o no ejecuta correctamente.
Calidad y rigor de la matriz de decisión tecnológica	1.3	Matriz de decisión ponderada con criterios específicos y justificados. Puntuaciones basadas en datos empíricos del benchmark. Justificación técnica sólida para cada puntaje asignado.	1.0	Matriz de decisión con criterios claros y ponderación apropiada. Puntuaciones basadas en resultados del benchmark con justificación básica. Decisión final fundamentada.	0.7	Matriz de decisión funcional, pero con criterios poco específicos o ponderación arbitraria. Conexión débil entre benchmark y puntuaciones.	0.3	Matriz de decisión superficial sin justificación clara de criterios o puntuaciones. Decisión no fundamentada en datos.	0	No presenta matriz de decisión o es completamente arbitraria.
Análisis técnico y	1.2	Análisis detallado del contexto de	1.0	Análisis apropiado	0.7	Análisis básico del	0.2	Análisis superficial	0	No presenta análisis

recomendaciones contextualizadas		emergencias 123 Bogotá. Recomendaciones específicas por volumen de datos. Trade-offs claramente explicados. Consideraciones de escalabilidad futura. Plan de migración tecnológica fundamentado.		del contexto con recomendaciones generales bien fundamentadas. Trade-offs identificados correctamente. Consideraciones básicas de escalabilidad.		contexto con recomendaciones genéricas. Trade-offs parcialmente identificados. Falta profundidad en consideraciones técnicas.		sin conexión clara al contexto. Recomendaciones genéricas sin fundamentación técnica.		técnico relevante o recomendaciones.
Documentación, presentación y reproducibilidad	1.0	Reporte comparativo (.xlsx) con métricas detalladas y gráficos profesionales. Notebook completamente documentado y reproducible. Screenshots informativos. Análisis de decisión (.docx) con formato ejecutivo. Código limpio y bien comentado.	0.8	Documentación clara con formatos apropiados. Notebook ejecutable con documentación básica. Entregables completos y organizados.	0.5	Documentación funcional, pero con formato básico. Notebook ejecutable con documentación mínima. Entregables presentes, pero poco organizados.	0.3	Documentación deficiente o incompleta. Dificultad para reproducir resultados. Formato inadecuado.	0	Documentación ausente o incomprensible. No reproducible.
Total	5		4		3		2.5		0	